Misión: Programa espacial en la NASA

Te han admitido para hacer tu FE como programador/a en la NASA. En tu primer día te asignan una tarea sencilla: revisar una pequeña parte del software que utilizan para registrar formas de vida extraterrestre detectadas por las sondas espaciales.

Una persona del equipo de desarrollo ha creado una clase llamada Alien, que guarda información básica sobre cada especie descubierta. Antes de usar este código en un sistema real, el equipo quiere asegurarse de que funciona correctamente.

Tu trabajo será comprobarlo utilizando pruebas unitarias con JUnit.

Para ello tendrás que investigar cómo:

- configurar un proyecto Java con soporte para tests
- crear y ejecutar pruebas con JUnit
- interpretar los resultados de los tests
- comprobar la cobertura del código

Si todo funciona correctamente, el sistema podrá utilizarse en futuras misiones espaciales.

Parte 1. Investigación y configuración del entorno

En Aules tenéis disponible la clase Alien.java con la que vais a trabajar.

Tu objetivo es investigar cómo crear y ejecutar pruebas unitarias en Java con JUnit utilizando IntelliJ IDEA.

Tareas a realizar:

1. Crear un nuevo proyecto llamado: `AlienTest`

   Recomiendo usar **Maven** para la creación del proyecto.

2. Añadir la clase Alien.java al proyecto.

3. Investigar cómo añadir **JUnit 5** al proyecto. Puedes hacerlo de dos maneras:

- utilizando las herramientas del propio IntelliJ
- añadiendo manualmente la dependencia correspondiente en el archivo pom.xml.

4. Investigar cómo se estructura un proyecto Java con Maven y responder:
   - ¿En qué carpeta se guarda el código fuente de la aplicación?
   - ¿En qué carpeta se guardan las pruebas unitarias?
5. Crear una clase de prueba para Alien llamada: *`AlienTest.java`*
6. Investigar cómo crear métodos de prueba utilizando la anotación: *`@Test`*

Parte 2. Primeros tests con la clase Alien

Ahora comienza la parte importante de la misión: comprobar que el sistema de registro de alienígenas funciona correctamente.

Crea varios tests para comprobar el funcionamiento de la clase **Alien**.

Puedes usar ejemplos como los siguientes:

```
Alien a1 = new Alien("Zorg", "MARS01");
Alien a2 = new Alien("Xenomorph", "LV426");
Alien a3 = new Alien("Zorg", "MARS01");
```

Test mínimos obligatorios

Debes implementar al menos **6 tests diferentes**.

Como mínimo debes comprobar:

- Que el constructor guarda correctamente el nombre.
- Que el constructor guarda correctamente el planetId.
- Que el método toString() devuelve el resultado esperado.
- Que dos objetos Alien con el mismo planetId se consideran iguales.

Tests adicionales obligatorios

También debes comprobar el funcionamiento de los **getters y setters**:

- Cambiar el nombre de un alien con setName() y comprobar que el cambio se guarda correctamente usando getName().
- Cambiar el planetId con setPlanetId() y comprobar que el cambio se guarda correctamente usando getPlanetId().

Para ello investiga cómo utilizar las siguientes **assertions de JUnit**:

```
assertEquals
assertTrue
assertFalse
assertNotNull
```

Ejecuta los tests desde IntelliJ y comprueba que todos pasan correctamente.

Buena práctica (Clen Code 😉)

En el equipo de la NASA intentan escribir código claro y fácil de entender.

Nombra los métodos de test de forma clara y descriptiva. Por ejemplo:

```
testConstructorStoresName()
testConstructorStoresPlanetId()
testAliensWithSamePlanetIdAreEqual()
```

Parte 3. Ejecución de pruebas y análisis

Después de crear tus tests para la clase Alien, ejecútalos desde el entorno y analiza los resultados obtenidos.

Responde a las siguientes preguntas:

1. ¿Qué significa que un test aparezca en **verde**?
2. ¿Qué significa que un test aparezca en **rojo**?
3. ¿Cómo se puede ejecutar **un solo test**?
4. ¿Cómo se ejecutan **todos los tests del proyecto**?

Análisis de un test con error

Para comprender mejor cómo funciona JUnit, crea también un test que falle intencionadamente. Por ejemplo, puedes comprobar un valor incorrecto como este:

```
Alien alien = new Alien("Zorg", "MARS01");
assertEquals("Xenomorph", alien.getName());
```

Al ejecutar este test, observa qué ocurre en IntelliJ y responde:

- ¿Qué valor aparece como **expected**?
- ¿Qué valor aparece como **actual**?
- ¿Cómo te ayuda ese mensaje a localizar el error?
- ¿Qué tendrías que cambiar para que el test pasara correctamente?

Una vez analizado el error, corrige el test y vuelve a ejecutarlo hasta que aparezca en **verde**.

Cobertura de código

Antes de enviar el software a una misión espacial, el equipo quiere saber qué partes del código están siendo comprobadas por los tests.

Investiga también cómo ejecutar las pruebas utilizando la opción `Run with Coverage`.

Responde:

- ¿Qué es la **cobertura de código**?
- ¿Qué significan los colores que aparecen en el editor?
- ¿Qué partes de tu clase Alien han sido ejecutadas por los tests y cuáles no?

Entrega

Debes entregar dos archivos en Aules:

1. Proyecto

Un archivo comprimido (.zip) con el **proyecto completo de IntelliJ** que incluya:

- la clase Alien.java
- la clase de tests AlienTest.java
- Todos los test implementados

2. Memoria breve

Un documento **PDF o Word** con:

- **Respuestas a las preguntas**

  Incluye las respuestas a todas las preguntas planteadas en la práctica (ejecución de tests, análisis de errores y cobertura de código).

- **Capturas obligatorias**

  Incluye las siguientes capturas de pantalla:

  - Estructura del proyecto donde se vean las carpetas src/main/java y src/test/java.
  - Código de la clase AlienTest con los tests implementados.
  - Ejecución de los tests donde se vea que todos aparecen en verde.
  - Test que falla intencionadamente con el mensaje de error (expected vs actual).
  - Cobertura de código ejecutada con *Run with Coverage*.

Las capturas deben permitir comprobar claramente que los tests se han ejecutado correctamente.